Umeå University

Master of Science in Computing Science (Robotics and Control), 30 ECTS

A Modular Modbus TCP Coordination Framework for AI-Based Industrial Inspection Software

Design and Evaluation for a Siemens S7-1200 and NorrSpect Integration Case

Ahmed Abuzaid

May 2026

Subject area:	Industrial automation, PLC systems, AI-based inspection
Umeå supervisor:	Ulrik Söderström
External partner:	NorrSpect AB
Primary technical focus:	Modbus TCP-based PLC/AI coordination
Artifact type:	Reusable PLC-side integration framework

Abstract

AI-based industrial inspection systems are increasingly used in production environments, yet their integration with programmable logic controller (PLC)-based automation is still often implemented as project-specific logic. This increases engineering effort, reduces reuse, and makes deployments fragile when inspection software exhibits variable latency, uncertain outputs, or restart-related faults. The problem becomes especially clear when a deterministic PLC-controlled workcell must coordinate with an external AI component that behaves less like a conventional industrial sensor and more like an asynchronous software service.

This thesis addresses that problem through a Modbus TCP-centered study grounded in the integration path between *NorrSpect* inspection software and a Siemens S7-1200 PLC. The aim is to design, implement, and evaluate a modular PLC integration framework that supports standardized, reusable, and scalable communication between AI-based inspection software and industrial automation equipment. The contribution is deliberately focused on the coordination layer rather than on inspection models or robot motion; the emphasis is instead on interface semantics, handshake design, timeout handling, restart recovery, observability, and software structure.

The research follows a design science methodology supported by a structured literature review, requirement derivation, artifact design, and controlled evaluation. A bench-informed Modbus baseline documented in the project material served as the starting point. Building on that baseline, the thesis develops a stronger coordination model based on explicit request identifiers, acknowledgement semantics, result validity, clear/reset handling, heartbeat monitoring, and reusable PLC state logic. Because repeatability and controlled fault injection mattered more than raw device benchmarking, the main empirical study was conducted in a software-in-the-loop environment that reproduces the relevant timing and register-exchange behavior of the intended PLC–AI interface.

The results show that the proposed framework improves correctness much more than speed, which is the appropriate trade-off for this class of integration problem. In

the nominal scenario, both the baseline and the proposed architecture achieved a successful-cycle p95 latency of 180 ms. Under delayed AI behavior, the successful completion rate increased from 87.7% to 94.7%, while unsafe stale-result acceptance fell from 5.8% to 0%. In the stale-result scenario, unsafe acceptance was reduced from 14.2% to 0%, and success improved from 71.3% to 84.3%. In the AI-restart scenario, the proposed design did not markedly increase successful completions, but it converted ambiguous failures into explicit fault detections, reduced timeouts, and eliminated stale acceptance. A scaling study further showed that a single shared polling client is adequate for one or two stations, but becomes a bottleneck at four or more stations if timeout parameters remain unchanged.

The thesis concludes that a Siemens S7-1200 PLC can serve as a reliable coordination layer for AI inspection software when Modbus TCP is supplemented with explicit application-level semantics rather than treated as a simple shared-register transport. The main engineering lesson is that protocol simplicity alone is not enough: result correlation, timeout-aware state transitions, and restart-safe reset logic are necessary if PLC-based automation is to absorb non-deterministic AI behavior without compromising workcell coordination. The thesis provides a reusable interface contract, a PLC-side architectural pattern, and a set of design principles applicable to future *NorrSpect* deployments and related industrial integration projects.

Acknowledgements

I would like to thank my supervisor at Umeå University and development team at *NorrSpect* AB for their support and the opportunity to work with such interestign topic at the intersection of industrial automation, software architecture, and AI-enabled manufacturing. Their support and feedback helped strengthen both the academic framing and the engineering result of the thesis. Finally, I thank my family and friends for their support throughout the thesis process.

I also acknowledge that generative AI tools were used during the preparation of this thesis to improve language refinement, consistency, and style. All technical decisions, analysis, and final checking of the text remain my own responsibility.

Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CPS	Cyber-Physical System
CPPS	Cyber-Physical Production System
DB	Data Block
DSR	Design Science Research
FB	Function Block
FC23	Modbus Function Code 23 (Read/Write Multiple Registers)
HMI	Human–Machine Interface
IEC	International Electrotechnical Commission
IIoT	Industrial Internet of Things
MBAP	Modbus Application Protocol header
MES	Manufacturing Execution System
PLC	Programmable Logic Controller
QC	Quality Control
RQ	Research Question
SCL	Structured Control Language
SFC	Sequential Function Chart
SQI	Smart Quality Inspection
TLS	Transport Layer Security

Contents

Abstract **i**

Acknowledgements **iii**

Abbreviations **iv**

1 Introduction **1**

1.1 Background and context 1

1.2 Practical problem and research problem 2

1.3 Aim 2

1.4 Research questions 3

1.5 Objectives 3

1.6 Scope and delimitations 4

1.7 Contributions 4

1.8 Thesis structure 4

2 Background and Theoretical Foundation **6**

2.1 Industry 4.0 and cyber-physical manufacturing 6

2.2 PLCs, IEC 61131-3, and the Siemens S7-1200 6

2.3 Modbus TCP fundamentals 7

2.4 Modbus on the S7-1200 8

2.5 Why AI inspection is different from a conventional sensor 8

2.6 Deterministic control and non-deterministic inference 9

2.7 State machines, handshakes, and recovery logic 9

2.8 Security considerations 10
2.9 Implications for the thesis design 10

3 Literature Review 12

3.1 Review strategy 12
3.2 Industry 4.0 and cyber-physical production systems 12
3.3 Industrial communication and the place of Modbus TCP 13
3.4 AI-based inspection and machine vision in manufacturing 14
3.5 PLC software structuring and modularity 14
3.6 Applied PLC–vision integration studies 15
3.7 Synthesis and research gap 15
3.8 Implication for artifact design 17

4 Research Methodology 18

4.1 Research stance 18
4.2 Research process 18
4.3 Why design science fits this thesis 19
4.4 Empirical basis and case logic 19
4.5 Evaluation strategy 20
4.6 Why software-in-the-loop evaluation was used 20
4.7 Experimental model parameters 21
4.8 Metrics 22
4.9 Reliability and validity 23
4.10 Ethical and practical considerations 23

5 Requirements and Reference Architecture 24

5.1 Requirement derivation 24
5.2 Architectural principle: separate transport from semantics 25
5.3 Responsibility allocation 26
5.4 Single-station and multi-station logic 27
5.5 Observability as a design requirement 27

6 Coordination Model and System Design 28

6.1 From register exchange to semantic contract 28

6.2 Documented baseline interface 28

6.3 Proposed semantic fields 29

6.4 Lifecycle of one inspection request 30

6.5 PLC-side state machine 31

6.6 Result semantics 32

6.7 Heartbeat and restart handling 33

6.8 Timeouts and bounded recovery 33

6.9 A scaling model for shared polling 33

6.10 Why the proposed design is better 34

7 Implementation 35

7.1 Implementation philosophy 35

7.2 Target hardware and software environment 35

7.3 PLC-side block decomposition 36

7.4 PLC coordination pseudocode 37

7.5 AI-side adapter behavior 38

7.6 Baseline and proposed implementation contrast 40

7.7 Instrumentation and logging 40

7.8 Implementation limits 41

8 Experimental Setup and Evaluation Procedure 42

8.1 Evaluation objective 42

8.2 Test scenarios 42

8.3 Injected delay and fault model 43

8.4 Outcome categories 44

8.5 Analytical recovery bound 44

8.6 Scaling procedure 44

8.7 Analysis procedure 45

9 Results **46**

9.1 Overview . . . 46

9.2 Nominal performance . . . 46

9.3 Delayed-AI scenario . . . 47

9.4 Communication-fault scenario . . . 47

9.5 Stale-result scenario . . . 48

9.6 AI-restart scenario . . . 48

9.7 Selected scenario metrics . . . 48

9.8 Restart detection and bounded recovery . . . 49

9.9 Scaling results . . . 49

10 Discussion **51**

10.1 Interpreting the main result . . . 51

10.2 What the thesis says about PLCs as coordination layers . . . 52

10.3 What the thesis says about Modbus TCP . . . 52

10.4 Why request identifiers matter so much . . . 52

10.5 Timeouts, faults, and engineering honesty . . . 53

10.6 Where PLC-based coordination is sufficient . . . 53

10.7 Where the limits begin . . . 54

10.8 Comparison with prior literature . . . 54

10.9 Practical implications for NorrSpect-style deployments . . . 55

10.10Limitations . . . 55

11 Conclusion and Future Work **57**

11.1 Conclusion . . . 57

11.2 Answers to the research questions . . . 57

11.3 Future work . . . 58

11.4 Final contribution statement . . . 59

A Current and Proposed Register Maps **60**

A.1 Current baseline map . . . 60

A.2 Proposed semantic map 60

B PLC State Transition Summary 62

C Experiment Matrix 63

List of Figures

5.1 Reference architecture used in the thesis. The key design decision is to place explicit coordination logic in the PLC layer instead of letting the protocol or the AI side implicitly define sequencing. 26

6.1 Nominal request lifecycle in the proposed coordination contract. The explicit request identifier is the mechanism that prevents stale-result acceptance. 31

6.2 PLC coordinator state machine used by the proposed framework. . . . 32

9.1 Latency distribution for successful cycles in the nominal and delayed-AI scenarios. 47

9.2 Configuration-derived recovery bound in the evaluation model. 49

9.3 Successful-cycle p95 latency as the number of stations sharing one polling client increases. 50

9.4 Successful cycle completion as the number of stations sharing one polling client increases. 50

List of Tables

3.1 Representative literature streams and their relevance to the thesis. . . . 16

4.1 Key evaluation parameters used in the software-in-the-loop experiments. 21

5.1 Derived requirements for the PLC integration framework. 25

6.1 Documented baseline Modbus register map used as the starting point for the thesis. 29

6.2 Proposed Modbus-oriented semantic contract for PLC–AI coordination. 30

7.1 Implementation environment targeted by the thesis artifact. 36

7.2 Recommended PLC-side function-block decomposition. 37

8.1 Experimental scenarios used in the evaluation. 43

9.1 Selected metrics for nominal, delayed, communication-fault, and stale-result scenarios. Latency values refer to successful cycles only. 48

9.2 Outcome breakdown for the AI-restart scenario. 49

9.3 Scaling results for the proposed architecture under nominal processing delay. 50

A.1 Baseline register map (documented starting point). 60

A.2 Proposed semantic register map. 61

B.1 State transition summary for the proposed PLC coordinator. 62

C.1 Experiment matrix used in the thesis. 63

Chapter 1

Introduction

1.1 Background and context

Manufacturing systems are becoming more connected through cyber-physical integration, sensors, networked production equipment, and data-based decision support. In this context, inspection is no longer only a final quality check after production. It becomes part of a wider digital control loop, where information about the product state can affect sequencing, traceability, production analysis, and sometimes machine behavior. This change is an important part of the Industry 4.0 vision described in the literature on cyber-physical manufacturing systems [1–3].

AI-based visual inspection illustrates this development clearly. Modern inspection systems can classify defects, report confidence values, provide richer metadata than conventional sensors, and adapt to more complex visual patterns than fixed rule-based systems [4, 5]. At the same time, they introduce a different integration challenge. The inspection system no longer behaves like a simple digital input or a threshold-based analog signal. Instead, it behaves like an asynchronous software subsystem whose response time can vary with workload, model complexity, runtime conditions, and fault state.

This thesis examines this challenge in the context of *NorrSpect*, a company developing AI-based industrial inspection software for PLC-controlled automation environments. The project implementation uses a Siemens S7-1200 testbed, Modbus TCP, and a Python-side simulator that supports both "sim mode" and "real mode" communication. The initial prototype showed that shared-register communication and Modbus Function Code 23 (FC23)-style exchange were technically possible; FC23 is introduced in section 2.3. However, it also showed that connectivity alone is not the main research issue. The more important question is how a deterministic PLC-centered

workcell should coordinate with a non-deterministic AI service in a way that is safe, repeatable, and reusable.

From a case-study perspective, *NorrSpect* is not treated as a statistical representative of all AI-inspection suppliers or as a unique edge case. It is used as a concrete example of a broader integration pattern: an external inspection service must exchange bounded results with PLC-owned machine sequencing. Similar PLC–vision integrations reported in the literature also rely on explicit interfaces, adapters, or state-based coordination to bridge vision software and automation logic [6, 7]. The general claims of the thesis should therefore be read as analytical generalization from this kind of PLC–AI coordination problem, not as a claim that every inspection product has the same timing or fault behavior.

1.2 Practical problem and research problem

In practical terms, many AI-based inspection deployments are still integrated with factory automation in an ad hoc, project-specific manner. Each new line, workcell, or customer installation may therefore require custom PLC logic, custom data mapping, and custom error handling. This increases commissioning effort and undermines reuse across deployments. In an industrial setting, the consequence is not only lower engineering productivity, but also weaker maintainability, poorer diagnosability, and higher long-term ownership cost.

The research problem focuses specifically on the coordination layer. PLC-based automation assumes cyclic execution, explicit signal exchange, and bounded timing. AI-based inspection introduces timing variability, richer outputs, uncertainty, and occasional restart or failure behavior. Existing literature shows that PLCs, machine vision systems, and industrial communication protocols can be integrated successfully [6, 7]. However, there is still limited empirical guidance on how to design a reusable PLC coordination layer for an external, asynchronous, and potentially fault-prone inspection system. In particular, Modbus-based studies often do not treat request correlation, timeout handling, stale-result protection, and restart recovery as central design concerns.

1.3 Aim

The aim of this thesis is to design, implement, and evaluate a modular PLC integration framework that enables standardized, reusable, and scalable communication between

AI-based inspection software and industrial automation systems through a Siemens S7-1200 PLC, using Modbus TCP as the primary communication mechanism.

1.4 Research questions

The thesis is guided by four research questions.

RQ1: To what extent can a Siemens S7-1200 PLC act as a reliable coordination and communication layer between *NorrSpect* AI-based inspection software and industrial automation equipment?

RQ2: How does Modbus TCP perform in the implemented S7-1200 integration path with respect to latency, jitter, robustness, recovery behavior, and scalability?

RQ3: How effectively can standardized handshakes and state-machine-based coordination manage AI-induced timing variation, communication delays, and fault scenarios over Modbus TCP?

RQ4: What architectural design principles can be generalized from the Siemens/Modbus testbed into a reusable PLC-side integration framework?

1.5 Objectives

To answer these questions, the thesis pursues the following objectives:

1. derive interface and architectural requirements from literature and case needs;
2. define a Modbus TCP coordination contract that goes beyond raw register exchange;
3. structure the PLC-side artifact as reusable functions, data blocks, and state logic suitable for a Siemens environment;
4. evaluate latency, jitter, fault handling, and restart behavior under controlled scenarios; and
5. distill design principles that can inform future deployments.

1.6 Scope and delimitations

The thesis is deliberately centered on computing-science and automation-integration questions. It does not optimize the AI inspection model, image acquisition hardware, or robot motion. It also does not claim to provide a safety-certified control architecture. Safety I/O, certified shutdown paths, and functional-safety validation remain outside scope. Cybersecurity is discussed only to the extent needed to understand the limitations of plain Modbus TCP and the implications for deployment.

A second delimitation concerns the empirical evidence. The project material documents a Siemens S7-1200 test path and a Modbus-based prototype, but the main experimental campaign presented in this thesis is software-in-the-loop rather than a full hardware benchmark. This choice was made to enable repeatable fault injection, controlled timing variation, and direct comparison between architectural variants. As a result, the thesis makes stronger claims about coordination logic and architectural behavior than about absolute device-level latency on specific hardware.

1.7 Contributions

The thesis makes four main contributions.

First, it defines a reusable Modbus-centered semantic contract for PLC–AI coordination, including request identifiers, acknowledgements, result validity, clear/reset handling, and heartbeat monitoring.

Second, it presents a PLC-side architectural pattern that separates communication handling, coordination logic, and workcell-level sequencing responsibilities.

Third, it provides comparative evidence showing how a stronger handshake and explicit state logic reduce unsafe outcomes under delay, stale-result, and restart conditions.

Fourth, it identifies the limits of a simple shared polling model and formulates practical design principles for scaling beyond a single inspection station.

1.8 Thesis structure

The remainder of the thesis is organized as follows. Chapter 2 introduces the technical background, including PLC programming structure, Modbus TCP, and the coordination challenge introduced by AI-based inspection. Chapter 3 positions the thesis in relation to existing literature. Chapter 4 presents the research design and evalua-

tion logic. Chapter 5 defines requirements and the reference architecture. Chapter 6 develops the coordination model and interface semantics. Chapter 7 describes the implementation logic. Chapter 8 explains the experimental setup and metrics. Chapter 9 reports the results. Chapter 10 interprets them. Chapter 11 concludes the thesis and proposes future work.

Chapter 2

Background and Theoretical Foundation

2.1 Industry 4.0 and cyber-physical manufacturing

The Industry 4.0 literature consistently describes modern manufacturing as the convergence of physical processes, embedded control, communication networks, and data-driven intelligence [1,3]. Lee et al. describe this convergence through a cyber-physical architecture in which data are acquired, contextualized, analyzed, and turned back into action [2]. Within such architectures, PLCs do not disappear. They remain the devices that own machine sequencing, interlocking, and deterministic signal handling, even when higher layers introduce analytics or machine learning.

That distinction matters for this thesis. The integration problem addressed here is not whether AI can improve inspection quality. It is how AI-generated inspection outcomes can be inserted into the deterministic rhythm of automation without breaking coordination. This places the thesis at the intersection of cyber-physical systems, industrial software engineering, and communication design.

2.2 PLCs, IEC 61131-3, and the Siemens S7-1200

A programmable logic controller (PLC) is an industrial controller that repeatedly reads input signals, executes programmed logic, and updates outputs in a cyclic real-time pattern. In a production cell, this scan-based behavior makes the PLC responsible for sequencing, interlocks, and deterministic machine reactions. PLCs remain the dominant execution platform for industrial sequencing and machine control because they

provide predictable scan-based execution, robust field integration, and maintainable program structures. IEC 61131-3 defines the standard programming languages and organizational principles commonly used in industrial control software [8]. Siemens states that the programming languages available in STEP 7 (TIA Portal) meet the requirements of IEC 61131-3 [9]. That includes the use of function blocks, functions, data blocks, and languages such as Ladder Diagram, Function Block Diagram, Structured Control Language, and SFC-related structures.

For this thesis, the most relevant Siemens-specific concept is the relationship between function blocks (FBs) and instance data blocks (DBs). An FB encapsulates reusable behavior and stores its persistent state in an instance DB. This is directly aligned with the objective of designing a modular and reusable PLC-side coordination framework. Rather than scattering request logic, timeout handling, and register access across machine code, the preferred pattern is to isolate the integration logic into dedicated FB/DB modules that can be instantiated and documented consistently.

The PLCopen structuring guidance reaches similar conclusions. It emphasizes modular decomposition, explicit interfaces, maintainability, and verifiability as core qualities of industrial control software [10–12]. Vogel-Heuser et al. show, from an industrial software-engineering perspective, that modularity in PLC-based automation remains unevenly applied in practice even though long-lived automated production systems need exactly that kind of maintainable structure [13]. This observation closely mirrors the industrial pain point behind the present thesis.

2.3 Modbus TCP fundamentals

Modbus is an application-layer protocol built around a client/server request–reply model [14]. In Modbus TCP, the application data unit consists of the Modbus Application Protocol (MBAP) header followed by the protocol data unit. The MBAP header contains a transaction identifier, protocol identifier, length field, and unit identifier. The protocol is intentionally simple: it is optimized for moving discrete values and registers between devices rather than for carrying rich semantic models.

That simplicity explains both its industrial popularity and its limitations. It is easy to implement, broadly supported, and well suited to small bounded data exchanges. Jaloudi [15] notes that Modbus retains practical value in IIoT environments because its message structure is efficient and familiar, even if broader architectures may require other protocols for eventing or higher-level semantics. Tapia et al. [16] likewise report comparatively low latency for Modbus in their empirical comparison of indus-

trial communication protocols for software monitoring. These characteristics make Modbus TCP a sensible primary protocol for the current thesis, particularly because it already matches the existing Siemens/*NorrSpect* test path.

Two details are especially important for the design developed in this work. First, Modbus itself does not provide request correlation semantics beyond the transaction level of the message exchange. If an application wants to know whether a result belongs to the current part, the previous part, or a restarted AI process, that logic must be implemented above the protocol. Second, Modbus Function Code 23 (FC23) permits reading and writing multiple registers in one transaction, with the write portion processed before the read portion [14]. This supports efficient cyclical data exchange and was already present in the baseline prototype.

2.4 Modbus on the S7-1200

On the Siemens S7-1200, Modbus TCP can be implemented using the standard `MB_CLIENT` and `MB_SERVER` instructions over the CPU's PROFINET interface, without additional hardware [17]. Siemens documents these instructions with parameters such as `REQ`, `CONNECT_ID`, `MB_MODE`, `MB_DATA_ADDR`, `MB_DATA_LEN`, `BUSY`, `ERROR`, and `STATUS`. This makes it possible to structure communication handling as a dedicated PLC service rather than as embedded ad hoc logic.

From an architectural perspective, that matters in two ways. First, the communication call itself can be encapsulated in a reusable function block. Second, diagnostic information returned by the Siemens instructions can be combined with application-level handshake information. The result is a cleaner separation between transport-level issues (connection errors, busy states, status codes) and application-level issues (late result, wrong request identifier, timeout, or heartbeat loss).

2.5 Why AI inspection is different from a conventional sensor

A conventional digital sensor typically produces a bounded, immediate, and semantically narrow output: high or low, on or off, reached or not reached. Even an analog sensor usually remains within a known sampling and conversion pattern. AI-based inspection is different. It may output a classification, a confidence value, an uncertainty marker, a defect category, an image identifier, or a processing error. More importantly,

the result can arrive with variable delay and may be affected by compute load or a software restart [4, 5].

The project material described this well: inspection results may include classifications, confidence values, richer payloads, and variable runtime. That observation is central to the thesis. The AI component should therefore not be modeled as a sensor that simply raises a bit. It should be treated as an asynchronous service that must be synchronized with the PLC-owned machine sequence.

2.6 Deterministic control and non-deterministic inference

The technical tension of the thesis can be stated plainly. PLC logic is deterministic in structure even if field signals vary. It is designed around cyclic scans, explicit conditions, and bounded transitions. AI inference is not deterministic in the same operational sense. Even when the same model is executed on the same data, response timing can vary due to queueing, compute contention, startup state, or fault recovery. When these two worlds meet, two classes of failure become likely.

The first class is *timing failure*. A valid result arrives too late for the current workpiece. The second class is *semantic failure*. A result is accepted, but it belongs to the wrong request or to an earlier cycle after a timeout or restart. The second class is more dangerous because it can appear superficially correct. If a PLC accepts a stale `PASS` or `FAIL` bit without correlation, the mistake can propagate silently into machine behavior.

This is why the thesis frames the core contribution around handshake semantics rather than raw connectivity. The real challenge is to create a coordination model that makes timing assumptions explicit, correlates results with requests, and provides defined recovery paths.

2.7 State machines, handshakes, and recovery logic

Industrial software guidance strongly favors explicit state-based structuring when systems interact with external devices or software modules [11, 12]. A state machine makes waiting, acknowledgement, timeout, and recovery paths visible in a way that scattered Boolean logic does not. It also improves diagnosability because each abnormal condition can be tied to a known state and transition.

The same argument applies to handshakes. A handshake is not just a convenience

for sequence ordering; it is an agreement about ownership and interpretation. In the present context, a robust handshake should distinguish at least the following concepts:

- the existence of a new inspection request,
- acknowledgement that the AI side has accepted that request,
- busy/processing state,
- validity of the inspection result,
- correlation between result and request,
- timeout and restart detection,
- clearing of consumed results, and
- explicit reset or re-synchronization after abnormal events.

Without these distinctions, a Modbus register map is simply a transport surface. With them, the register map becomes a coordination contract.

2.8 Security considerations

Although security is not the experimental focus of this thesis, it must be acknowledged because it shapes realistic industrial deployment. Plain Modbus TCP does not provide built-in encryption or strong authentication. The Modbus Organization's security specification addresses this by defining a TLS-protected variant that uses port 802 and certificate-based security mechanisms [18]. In practice, however, many installations still rely on network segmentation, VPNs, firewalls, and trust assumptions around internal plant networks.

For this reason, the thesis treats cybersecurity as a deployment consideration rather than a claim of completeness. The proposed framework improves functional correctness and coordination safety, but it does not by itself solve network-level trust, certificate management, or defense against malicious traffic.

2.9 Implications for the thesis design

The background above leads to three implications that shape the rest of the thesis.

First, Modbus TCP should be treated as the transport substrate rather than the full interface specification. The necessary semantics must be designed above it.

Second, the PLC-side artifact should be modular and state-based, consistent with IEC 61131-3 structuring and Siemens FB/DB patterns.

Third, the evaluation must include delayed, stale, restart, and fault scenarios. If the artifact only works in the nominal case, it has not solved the real integration problem.

Chapter 3

Literature Review

3.1 Review strategy

The literature review was conducted as a structured narrative review rather than a fully systematic review. This choice follows from the purpose of the chapter. The aim is not to catalogue every publication on PLCs, machine vision, or industrial communication, but to position the thesis at the intersection of the main knowledge streams that shape the research problem: Industry 4.0 and cyber-physical production systems, industrial communication protocols, PLC software architecture, AI-based inspection, and applied PLC–vision integration studies.

Searches were conducted iteratively using combinations of terms such as *PLC AI inspection*, *machine vision PLC integration*, *Modbus TCP latency*, *IEC 61131-3 modularity*, *state machine industrial automation*, and *cyber-physical production systems*. Priority was given to peer-reviewed journal and conference papers from 2014–2026, together with primary standards and official vendor documentation where protocol or platform behavior required authoritative grounding. This time window captures the period in which Industry 4.0 and practical AI-enabled quality inspection became established research topics.

3.2 Industry 4.0 and cyber-physical production systems

Monostori characterizes cyber-physical production systems as one of the key foundations of the fourth industrial revolution, emphasizing the convergence of computation, communication, and manufacturing technology [1]. Colombo et al. develop a simi-

lar view, describing industrial cyber-physical systems as a backbone technology that enables connected, responsive, and collaborative industrial environments [3]. Lee et al. go further by proposing the influential 5C architecture, which links connection, conversion, cyber, cognition, and configuration across manufacturing systems [2].

These works are important because they provide the macro-level justification for the thesis. They explain why production data, inspection, and distributed control should be considered together rather than in isolation. At the same time, they stop at a level of abstraction that is too high for the present research problem. They do not explain how a PLC should manage a delayed inspection response, distinguish a stale result from a current one, or recover from an AI-side restart over a concrete industrial protocol. Their relevance is therefore foundational rather than directly prescriptive.

3.3 Industrial communication and the place of Modbus TCP

The industrial communication literature makes two points that matter directly to this thesis. First, communication protocols differ not only in technical features but also in software complexity, interoperability, and data-model richness. Second, protocol choice alone does not resolve application-level coordination problems.

Wollschlaeger et al. describe industrial communication as an area under strong transformation, driven by the need to integrate field-level automation with richer information exchange across Industry 4.0 systems [19]. Jaloudi's comparative study of IIoT communication protocols similarly shows that Modbus remains valuable because of its efficient and simple message structure, but also notes that complementary protocols may be required in broader architectures [15]. Tapia et al. provide particularly relevant evidence: in their empirical study of industrial communication protocols for software monitoring, Modbus showed the best latency figures among the tested alternatives, but the paper also emphasized development complexity and application context [16].

What this literature does *not* provide is a complete answer to the thesis problem. Tapia et al. evaluate performance and software complexity, not AI-induced coordination faults. Jaloudi discusses protocol roles at an IIoT level, not request/result correlation between a PLC and an external inspection service. The implication is important: Modbus TCP is a defensible engineering choice for the present work, but the thesis contribution has to be built above the protocol rather than searched for inside it.

3.4 AI-based inspection and machine vision in manufacturing

The literature on AI-enabled quality inspection is now broad and mature enough to show clear technical viability. Sundaram and Zeid propose a smart quality inspection approach combining deep learning with a shop-floor deployment tool, arguing that a practical inspection system must address not only model accuracy but also deployment constraints and operator interaction [4]. Reichenstein et al. focus on implementing machine-vision-based quality inspection in production and stress that industrial planning and execution remain case-study-heavy and methodologically fragmented [20]. More recently, Patrashko and Gurau review machine-learning-powered vision for robotic inspection and show that many published systems achieve high performance in controlled settings while still facing important deployment barriers and pilot-scale limitations [5].

This stream is crucial because it explains why richer inspection outputs and timing variation must be taken seriously. However, most of this literature evaluates inspection capability, defect detection performance, or deployment methodology. The PLC is often implicit in the background or treated as a generic integration endpoint. The practical coordination layer between AI software and workcell automation is therefore underdeveloped in these works.

3.5 PLC software structuring and modularity

The literature on PLC software engineering is particularly valuable for the present thesis because it shifts attention away from communication as mere data movement and toward software qualities such as maintainability, decomposition, and reuse. PLCopen's structuring documents make the case that IEC 61131-3 should be used in a disciplined way to create understandable, maintainable, and reusable control software [10–12]. Siemens' own documentation is compatible with that position through its emphasis on function blocks, instance data blocks, and explicit communication instructions [9, 17].

Vogel-Heuser et al. provide a valuable industry-facing complement. Their study of modularity and architecture in PLC-based software for automated production systems shows that long-lived industrial systems need stronger modularity and architectural discipline than current practice often provides [13]. This insight maps directly onto the *NorrSpect* problem. If each AI inspection deployment requires new PLC glue logic, then the integration approach is architecturally immature even when the line "works".

The limitation of this literature is that it rarely deals with AI-induced non-determinism explicitly. It tells us how PLC software *should* be structured, but not exactly what semantic contract is required when an external inspection component may time out, restart, or return results late.

3.6 Applied PLC–vision integration studies

The most directly adjacent literature to this thesis is the small but growing body of work that integrates machine vision with PLC-based automation. Maślanka et al. present a vision–PLC integration framework for scalable quality inspection in Industry 4.0 and report reliable synchronization in an applied case study [6]. Almtireen et al. integrate a PLC-controlled conveyor system with AI-enhanced vision for waste sorting, showing that PLC-based automation and AI-driven vision can be combined in a coherent prototype [7].

These papers are important because they confirm the industrial relevance of PLC–vision coupling and move beyond purely conceptual architectures. However, they also reveal the main gap that motivates this thesis. The systems are application-specific. Their primary contribution is that the prototype works for the task at hand. They are much less explicit about reusable semantic contracts, restart-safe handshake design, or the trade-off between simplicity and correctness in a register-based protocol such as Modbus TCP.

3.7 Synthesis and research gap

Taken together, the literature suggests five conclusions.

First, Industry 4.0 and cyber-physical manufacturing clearly motivate tighter integration between inspection, control, and software systems.

Second, Modbus TCP remains an industrially relevant and technically efficient transport choice, especially when simplicity, availability, and latency matter.

Third, AI-based inspection systems are not “just sensors”; they behave as asynchronous and sometimes uncertain software components.

Fourth, PLC software architecture literature strongly supports modular, reusable, and state-based structuring.

Fifth, applied PLC–vision studies show feasibility but remain mostly case-specific and thin on reusable coordination design.

This yields the central research gap addressed in the thesis: *there is limited empirical guidance on how a reusable PLC coordination layer should be designed and evaluated when an external AI inspection system communicates over Modbus TCP and exhibits non-deterministic timing, stale-result risk, and restart-related faults.*

Table 3.1 summarizes the literature streams that lead to this gap.

Table 3.1: Representative literature streams and their relevance to the thesis.

Stream	Representative works	Main contribution	Limitation relative to this thesis
Industry 4.0 and CPS	[1–3]	Positions manufacturing as a cyber-physical, connected, data-driven environment.	Too high-level to specify PLC/AI coordination semantics.
Industrial communication	[15, 16, 19]	Explains protocol trade-offs and provides evidence that Modbus remains competitive in latency-sensitive contexts.	Does not address stale-result protection, restart handling, or AI-specific handshakes.
AI inspection in manufacturing	[4, 5, 20]	Demonstrates the technical value and deployment challenges of AI-based inspection.	Focuses on model capability and deployment, not PLC orchestration.
PLC software structuring	[10, 11, 13]	Supports modular, state-based, maintainable PLC software architecture.	Does not evaluate coordination with external AI services under timing uncertainty.
Applied PLC–vision integration	[6, 7]	Shows that PLCs and vision/AI can be integrated successfully in practical cases.	Mostly application-specific; limited reusable framework guidance.

3.8 Implication for artifact design

The literature review informs the artifact in a precise way. It suggests that the right contribution is neither a new inspection algorithm nor a broad protocol survey. Instead, the contribution should be a *Modbus-centered semantic and architectural framework* that translates PLC software-structuring principles into a practical coordination layer for external AI inspection.

That implication drives the rest of the thesis: transport simplicity is preserved, but coordination semantics are strengthened.

Chapter 4

Research Methodology

4.1 Research stance

This thesis is best understood as design science research with case-informed evaluation. Design science is appropriate when the contribution takes the form of an artifact—here, a reusable PLC-side coordination framework—and when the purpose is not merely to describe an industrial phenomenon, but to improve it through purposeful design [21, 22]. The artifact is therefore developed against a real problem class rather than as an isolated technical exercise.

The study is also anchored in a concrete integration path involving *NorrSpect*, a Siemens S7-1200 PLC, and a Modbus TCP interface. Following Flyvbjerg, the value of such a case lies less in statistical representativeness than in analytical usefulness: a well-chosen case can illuminate a broader class of problems when the reasoning from case to contribution is made explicit [23].

4.2 Research process

The research process follows the logic proposed by Peffers et al. for design science research [22] and is organized in six stages:

1. **Problem identification and motivation.** The industrial integration problem was defined from the case context and sharpened through the literature review.
2. **Definition of objectives.** Requirements were formulated around reliability, reuse, robustness, and observability.

3. **Design and development.** A stronger coordination contract and PLC-side architecture were designed on top of the documented Modbus baseline.
4. **Demonstration.** The artifact was expressed as a Siemens-oriented PLC architecture and exercised in a software-in-the-loop environment that mirrors the intended Modbus exchange pattern.
5. **Evaluation.** Baseline and proposed designs were compared under nominal, delayed, stale, fault, restart, and scaling scenarios.
6. **Communication.** The design principles, results, and limitations are documented in this thesis.

4.3 Why design science fits this thesis

Design science is a strong methodological fit for two reasons.

First, the thesis develops a practical artifact with a clear engineering purpose, rather than only presenting a descriptive classification. The contribution includes the meaning of the interface signals, the state transitions, the structure of the function blocks, and the measured behavior of the system during controlled fault-injection tests.

Second, the evaluation criteria are utility-oriented while remaining academically grounded. The artifact is assessed through latency, jitter, success rate, unsafe outcome rate, and recovery behavior. These metrics are not arbitrary industrial preferences; they operationalize the research questions.

4.4 Empirical basis and case logic

The empirical basis of the thesis has three layers.

The first is the documented case context: *NorrSpect* inspection software, a Siemens S7-1200 testbed, and a Modbus TCP baseline already using an FC23-style shared-register exchange. This provided a realistic starting point and kept the research problem anchored in an actual industrial integration path.

The second is literature-based requirement derivation. Work on PLC modularity, industrial communication, and AI inspection was used to challenge the initial project framing and refine the artifact requirements.

The third is a controlled software-in-the-loop evaluation environment. This environment reproduces the interface conditions that matter for the intended PLC–AI inte-

gration: scan-driven PLC execution, cyclical Modbus polling, variable AI processing delay, communication drops, stale results, and restart events.

The case should therefore be understood as an *instrumental case*. It grounds the artifact in a real industrial need while still allowing the evaluation to focus on coordination mechanisms with broader relevance.

4.5 Evaluation strategy

The evaluation is comparative. Two architectural variants are examined:

- a BASELINE architecture representing a simple shared-register exchange without request identifiers or robust stale-result protection; and
- a PROPOSED architecture with explicit request identifiers, acknowledgement semantics, result validation, clear/reset handling, and heartbeat monitoring.

Both variants are exposed to the same workload and fault scenarios. The goal is not to argue that one protocol is universally superior, but to determine whether explicit coordination semantics improve reliability and diagnosability without introducing an unacceptable latency penalty.

4.6 Why software-in-the-loop evaluation was used

A purely hardware-based campaign would have offered greater physical realism, but not necessarily the most useful evidence for this thesis. The research questions concern coordination under delay, stale data, and restart conditions. Those behaviors are easier to reproduce, isolate, and compare in a controlled software-in-the-loop environment than in a limited set of bench runs.

The choice also reflects a common constraint in industrial thesis work: hardware access windows are finite, while systematic fault injection is difficult to standardize on a live bench. The simulator therefore complements rather than replaces the Siemens test path. It should be understood as a repeatable analytical environment calibrated to the intended interface model, not as a claim of cycle-accurate emulation of every detail of a Siemens CPU.

The simulated AI block also deliberately excludes image-classification errors, data-quality problems, unexpected AI-side software faults, and plant-level disturbances.

This isolation is useful for comparing coordination designs, but it means that production reliability must still be verified through hardware-in-the-loop and real-system testing.

4.7 Experimental model parameters

The software-in-the-loop environment uses discrete-time simulation with parameters chosen to approximate a modest industrial coordination loop while keeping scenario comparisons repeatable. The parameter values used in the model are summarized in Table 4.1.

Table 4.1: Key evaluation parameters used in the software-in-the-loop experiments.

Parameter	Value	Role
Discrete simulation step	5 ms	Time resolution for the model
PLC scan period	10 ms	PLC state evaluation interval
Base Modbus polling interval per station	20 ms	Nominal FC23-style exchange period
Request timeout	600 ms	Upper bound before timeout handling
Baseline recovery delay	120 ms	Recovery after timeout without explicit correlation
Proposed recovery delay	60 ms	Recovery after explicit reset/clear
Scenario repetitions	600 cycles	Comparative scenario evaluation
Scaling repetitions	400 cycles	Multi-station nominal scaling evaluation
Random seed	42	Repeatability of stochastic events

The absolute values should not be read as universal PLC constants. Their purpose is to provide a repeatable environment in which differences between coordination designs become visible. The scan and polling periods make request/response ordering visible at a PLC-like time scale; the 600 ms timeout leaves room for nominal completion while still exposing delayed, stale, and restart cases; and the recovery delays model the difference between implicit cleanup and explicit clear/reset handling. The repetition counts were chosen to make percentage differences stable enough for comparison while keeping the experiment traceable.

4.8 Metrics

The main quantitative metrics are defined as follows.

For cycle i, end-to-end latency is defined as

$$L_i = t_i^{\text{accept}} - t_i^{\text{request}}, \tag{4.1}$$

where t_i^{request} is the time at which the PLC issues a new inspection request and t_i^{accept} is the time at which the PLC accepts a result or fault outcome for that cycle.

The successful completion rate is

$$S = \frac{N_{\text{success}}}{N_{\text{total}}}, \tag{4.2}$$

and the timeout rate is

$$T = \frac{N_{\text{timeout}}}{N_{\text{total}}}. \tag{4.3}$$

For unsafe outcomes, the most relevant quantity is stale-result acceptance:

$$U = \frac{N_{\text{stale accept}}}{N_{\text{total}}}. \tag{4.4}$$

Timing variability is reported as the population standard deviation of successful latencies,

$$J = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(L_i - \bar{L})^2}, \tag{4.5}$$

where n is the number of successful cycles.

The primary latency statistic is p95 successful-cycle latency. In this thesis, p95 means the latency value below which 95% of successful-cycle latencies fall after timeout and

fault cycles have been excluded from the latency sample and counted separately in the outcome metrics. It was selected as the headline latency statistic because it captures near-tail response-time behavior relevant to PLC waiting limits while being less dominated by isolated extreme observations than p99 in finite scenario runs. The p50 statistic was used as a median reference for the latency distribution, and p99 was retained as a tail diagnostic, especially in the scaling experiment where serialized polling can create near-timeout outliers.

4.9 Reliability and validity

Reliability, in the narrow experimental sense, was supported through deterministic parameterization and a fixed random seed. All comparative scenarios use the same simulation rules, timing model, and number of repetitions.

Construct validity was addressed by aligning the metrics with the research questions. Because the thesis is concerned with coordination quality rather than image-classification accuracy, measures such as stale-result acceptance, explicit restart-fault detection, and recovery behavior are more informative than model precision.

Internal validity is strengthened by the direct baseline/proposed comparison under identical scenarios. The main threat is model simplification: the simulator does not reproduce the full network stack, CPU load variation, or the detailed timing behavior of a physical Siemens installation.

External validity is best framed as analytical rather than statistical generalization. The exact latency values should not be generalized indiscriminately. The design principles, however, are likely to transfer to other PLC-centered integrations in which external AI inspection must be synchronized with deterministic machine sequencing.

4.10 Ethical and practical considerations

The thesis does not involve personal data, human subjects, or sensitive operator information. The evaluation relies on synthetic workpiece cycles and simulated inspection timing. Industrial confidentiality is respected by focusing on interface structure and coordination semantics rather than exposing product-specific inspection models or proprietary customer workflows.

Chapter 5

Requirements and Reference Architecture

5.1 Requirement derivation

Requirements were derived from three sources: the industrial case, the literature review, and the failure modes implied by deterministic PLC control interacting with asynchronous AI software. The resulting requirements, listed in Table 5.1, reflect both immediate engineering needs and the broader objective of producing a reusable framework.

Table 5.1: Derived requirements for the PLC integration framework.

ID	Requirement	Motivation
R1	The PLC interface shall support explicit request/response synchronization.	Prevents ambiguous sequencing and implicit timing assumptions.
R2	Each accepted result shall be correlatable to the request that triggered it.	Prevents stale or duplicated results from being consumed.
R3	The interface shall expose timeout and recovery paths.	AI latency and communication faults must not deadlock the workcell.
R4	The interface shall distinguish valid inspection outcomes from communication or software faults.	A failed inspection is not the same thing as a missing or invalid result.
R5	The PLC-side implementation shall be modular and reusable.	Supports deployment across multiple stations and customer projects.
R6	The design shall remain compatible with Modbus TCP and Siemens S7-1200 implementation patterns.	Matches the current industrial integration path and avoids unnecessary protocol scope expansion.
R7	The framework shall provide observability through counters, status values, and diagnosable states.	Necessary for commissioning, maintenance, and fault analysis.
R8	The design shall scale at least conceptually from a single station to multiple stations.	Reuse and deployment value depend on more than one-off integration.

5.2 Architectural principle: separate transport from semantics

The core architectural principle of the thesis is that Modbus TCP should carry a semantic contract rather than be mistaken for one. A Modbus register map by itself only defines where bits and words are stored. It does not define when a result is safe

to consume, whether it belongs to the current part, or how both sides recover after a timeout.

This principle leads to the layered reference architecture shown in figure 5.1, in which transport, coordination, and machine sequencing are distinct concerns.

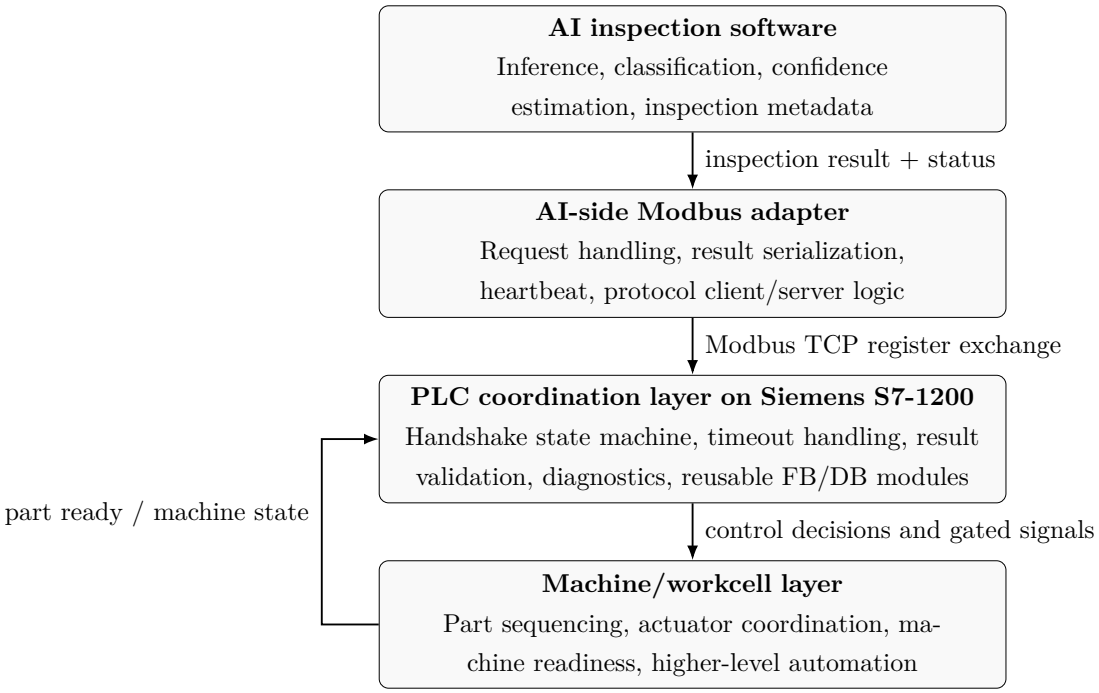


Figure 5.1: Reference architecture used in the thesis. The key design decision is to place explicit coordination logic in the PLC layer instead of letting the protocol or the AI side implicitly define sequencing.

5.3 Responsibility allocation

The architecture above deliberately assigns different responsibilities to different layers.

The *AI inspection software* owns inference, classification, and inspection-specific outputs. It should not own workcell sequencing.

The *AI-side adapter* translates between internal inspection outputs and the external Modbus register contract. It can run in simulation mode or real integration mode, but in both cases it should honor the same handshake semantics.

The *PLC coordination layer* owns request issuance, acknowledgement tracking, timeout enforcement, restart detection, and the decision of when a result is safe to release into the machine sequence.

The *machine/workcell layer* remains the place where machine readiness, part transport, station interlocks, and actuator logic are controlled.

This division avoids two opposite mistakes. The first mistake is to make the PLC a passive data mirror with no authority over result acceptance. The second is to push workcell coordination into the AI application. The proposed architecture rejects both.

5.4 Single-station and multi-station logic

The project materials already indicate interest in both single- and multi-station use. A single-station case is the natural starting point because it minimizes concurrency and makes fault behavior easier to interpret. However, reusability requires that the architecture be extendable.

At a semantic level, the framework scales naturally: each station can have its own request identifiers, result registers, diagnostics, and FB instance. At a communication level, scaling is more demanding. If multiple stations share a single polling client or a single serialized communication loop, effective poll interval grows with the number of stations. The experiments later in the thesis show that this can become a bottleneck if timeouts are not adjusted or if communication remains strictly serialized.

5.5 Observability as a design requirement

Observability is often treated as an afterthought in industrial integration, but that is a mistake when asynchronous software is involved. A reusable framework should expose enough internal state to answer basic commissioning questions:

- Did the AI side acknowledge the current request?
- Is the AI side still alive?
- Was the result late, stale, or explicitly faulted?
- How many timeouts have occurred?
- Which state is the PLC coordinator currently in?

The proposed architecture therefore includes counters, heartbeat fields, and fault codes in addition to the core handshake bits.

Chapter 6

Coordination Model and System Design

6.1 From register exchange to semantic contract

The documented baseline prototype already used a shared register map and FC23-style read/write exchange. That baseline is a valid starting point because it proves that the Siemens/Modbus path is technically feasible. However, it is still best characterized as a data-exchange scheme rather than a full coordination contract. The next step is to make the meaning of the exchanged data explicit.

The thesis therefore distinguishes between two interface levels:

1. the *transport level*, which covers Modbus messages, addresses, and read/write behavior; and
2. the *semantic level*, which defines request ownership, acknowledgment, result validity, restart recovery, and fault interpretation.

6.2 Documented baseline interface

The project material describes a compact baseline map in which one register block is read and another is written. The exact semantics are limited, but the baseline is useful because it reflects the kind of simple exchange that often emerges first in practical integration work. Table 6.1 summarizes this documented starting point.

Table 6.1: Documented baseline Modbus register map used as the starting point for the thesis.

Registers	Meaning in the baseline prototype
0–9	Read block: status, pieces inspected, batch identifier, serial number, defect rate, and reserved fields.
10–19	Write block: machine status bits and reserved fields. Register 10 includes at least Stop, Emergency Stop, and Ready bits.

This structure is intentionally simple and remains useful for backward compatibility. The problem is not the existence of registers; it is the absence of explicit lifecycle semantics around those registers. A robust integration must answer questions such as: Which part does this result belong to? Has the AI side accepted the request? Has the result already been consumed? Is the AI side alive but busy, or disconnected and silent?

6.3 Proposed semantic fields

The Proposed framework keeps the Modbus approach but upgrades the register semantics. Table 6.2 shows the conceptual data contract. The exact numeric register addresses can be adapted to project constraints; what matters is the semantics and separation of responsibilities.

Table 6.2: Proposed Modbus-oriented semantic contract for PLC–AI coordination.

Register	Field	Purpose
0	`req`	PLC requests a new inspection cycle.
1	`req_id`	Monotonic identifier for the current request.
2	`clear`	PLC signals that the current result has been consumed and the interface may be cleared.
3	`reset`	PLC requests forced re-synchronization after timeout or abnormal state.
4	`hb_plc`	PLC heartbeat toggled cyclically.
10	`ack`	AI side acknowledges the current request.
11	`ack_id`	Identifier of the acknowledged request.
12	`busy`	AI side is processing the current request.
13	`result_valid`	Result fields contain a valid inspection outcome.
14	`result_id`	Identifier of the request to which the result belongs.
15	`result_code`	Encoded inspection outcome (for example PASS/FAIL/UNCERTAIN/ERROR).
16	`confidence`	Optional confidence or score field, scaled as needed.
17	`hb_ai`	AI-side heartbeat toggled cyclically.
18	`fault_code`	Explicit abnormal condition such as timeout, restart, or protocol inconsistency.
19	Reserved	Available for station-specific extensions.

The conceptual change is more important than the added field count. The baseline asked "what values are in the registers?" The proposed design asks "what part of the coordination lifecycle does each value belong to?"

6.4 Lifecycle of one inspection request

The request lifecycle in the Proposed design is shown conceptually in figure 6.1. The PLC remains the sequence owner. The AI side may only produce a result after acknowledging a specific request.

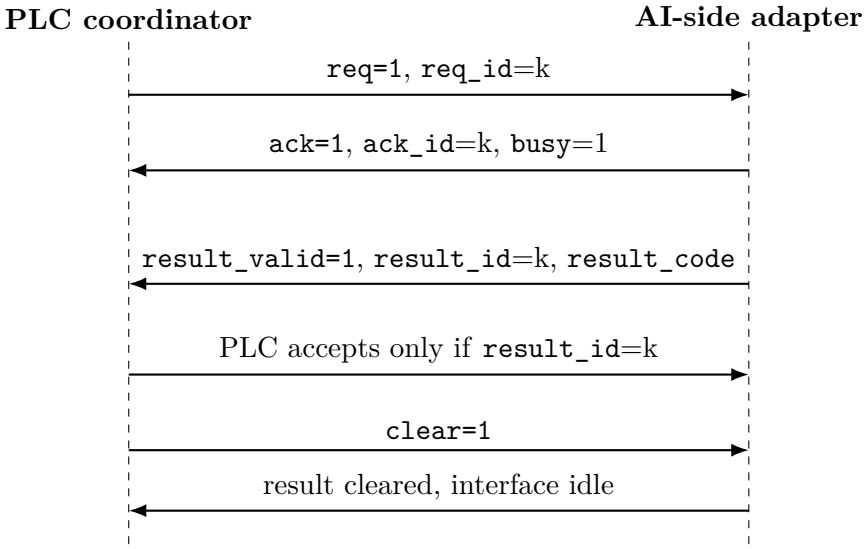


Figure 6.1: Nominal request lifecycle in the proposed coordination contract. The explicit request identifier is the mechanism that prevents stale-result acceptance.

The nominal lifecycle is therefore:

1. a part arrives and the PLC raises `req` with a new `req_id`;
2. the AI side latches the request, acknowledges it, and enters `busy`;
3. when processing completes, the AI side publishes a valid result together with `result_id`;
4. the PLC accepts the result only if `result_id` equals the current `req_id`;
5. the PLC asserts `clear`; and
6. the AI side clears result-related fields and both sides return to idle.

6.5 PLC-side state machine

The semantic contract above is enforced by the PLC-side state machine shown in figure 6.2. The logic is intentionally simple enough to fit an industrial FB but explicit enough to remain diagnosable.

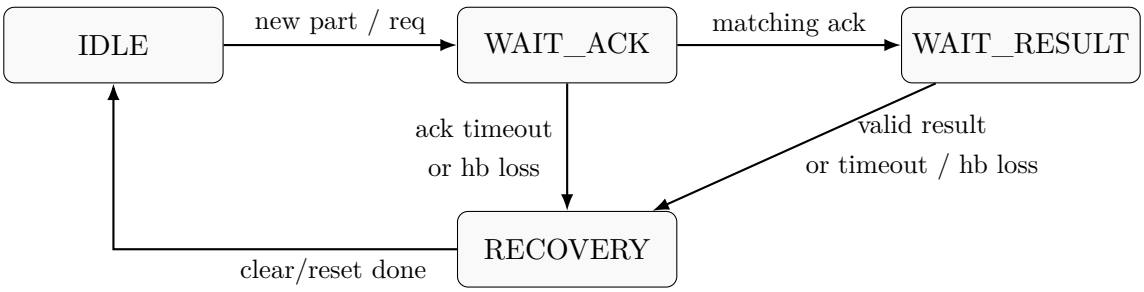


Figure 6.2: PLC coordinator state machine used by the proposed framework.

The state machine has four main states:

IDLE The coordinator is ready for a new workpiece.

WAIT_ACK
A request has been issued and the PLC waits for the AI side to acknowledge ownership of it.

WAIT_RESULT
The request was acknowledged and the PLC now waits for a matching valid result.

RECOVERY
The coordinator clears or resets interface state after either successful completion or abnormal termination.

This state model is intentionally compact. Industrial maintainability benefits from a state machine that an engineer can fully understand during commissioning without the need to decode deeply in Boolean conditions.

6.6 Result semantics

A second design decision is to separate inspection *outcome* from communication *health*. The proposed result codes include at least `PASS`, `FAIL`, `UNCERTAIN`, and `ERROR`. These reflect inspection meaning. By contrast, timeout, restart, or heartbeat loss are expressed through state transitions and fault codes.

This distinction is important. An uncertain inspection result is still a valid inspection result and should lead to a defined machine response, such as manual review or recapture. A communication timeout is different: it means the workcell lacks a trustworthy result and must handle that as a coordination failure.

6.7 Heartbeat and restart handling

The AI side can fail in ways that are not visible through result registers alone. If the process restarts or the communication loop freezes while the PLC is waiting, the absence of progress may look identical to a slow inspection. The proposed contract therefore introduces a lightweight heartbeat on both sides.

The PLC toggles `hb_plc` cyclically. The AI side toggles `hb_ai`. If the PLC observes no heartbeat change within the watchdog window, it can classify the event as a restart or communication fault rather than waiting indefinitely for a result. This does not replace network-level diagnostics, but it gives the application-level coordination layer its own liveness signal.

6.8 Timeouts and bounded recovery

Timeout behavior is unavoidable when integrating non-deterministic software into deterministic sequencing. The question is not whether a timeout will ever happen, but whether it leaves the interface in a safe and diagnosable state.

In the baseline model, a timeout simply drops the request and waits before allowing the next part. Because there is no request correlation, a late result can remain visible and be accepted in the next cycle.

In the proposed model, a timeout triggers an explicit recovery path: the request is dropped, `clear` and `reset` are asserted, and the PLC returns to idle only after the interface has been re-synchronized. This is the mechanism that removes stale-result acceptance in the experimental scenarios.

6.9 A scaling model for shared polling

For a single station, the Modbus polling interval can remain short and predictable. When n stations share one serialized polling client, the effective poll allowance grows approximately as

$$T_{\text{poll}}(n) \approx n \cdot T_0, \tag{6.1}$$

where T_0 is the base polling interval per station.

This relation is intentionally simple, but it highlights why the same coordination logic can behave differently at larger scale. As n grows, acknowledgment visibility, result visibility, and heartbeat visibility are all delayed. A framework that is perfectly safe

for a single station may therefore require different timeout values or parallel communication instances when scaled.

6.10 Why the proposed design is better

The improvement over the baseline is not that it uses more registers. The improvement is that it makes failure modes explicit and recoverable. The three most important changes are:

1. **Request identifiers** eliminate ambiguity about which part a result belongs to.
2. **Clear/reset semantics** ensure that consumed or faulted state is actively removed instead of left to implicit decay.
3. **Heartbeat monitoring** turns silent waiting into diagnosable liveness loss.

These mechanisms are modest in implementation cost, but they fundamentally change the safety and diagnosability of the Modbus integration.

Chapter 7

Implementation

7.1 Implementation philosophy

The implementation described in this chapter is deliberately split between a Siemens-oriented PLC design and a Python-based AI-side module. This mirrors the case context and follows a sound architectural principle: the PLC should own sequencing and coordination, while the AI side should own inspection logic and expose its outputs through the agreed interface contract.

7.2 Target hardware and software environment

The target industrial environment is a Siemens S7-1200 PLC programmed in TIA Portal and connected to an AI-side application through Modbus TCP. The project materials also describe a Python-based simulator capable of working in both simulation and real integration modes, which provided a practical basis for testing connectivity before a full connection to the industrial inspection software. Table 7.1 summarizes the main implementation environment.

Table 7.1: Implementation environment targeted by the thesis artifact.

Component	Role in the framework
Siemens S7-1200 PLC	Coordination layer, Modbus communication endpoint, state-machine execution.
TIA Portal / STEP 7	PLC program development environment with IEC 61131-3 compliant languages and Siemens communication blocks.
`MB_CLIENT` / `MB_SERVER`	Siemens instructions used to implement Modbus TCP communication.
Python AI-side adapter	Bridges inspection software outputs to the Modbus register contract; supports simulation and real integration modes.
Software-in-the-loop harness	Controlled experimentation under delay, stale-result, communication-fault, and restart scenarios.

7.3 PLC-side block decomposition

A reusable PLC implementation should not bury coordination logic inside station-specific code. The recommended decomposition is shown in Table 7.2.

Table 7.2: Recommended PLC-side function-block decomposition.

Block	Responsibility	Why it is separated
`FB_CommModbus`	Calls `MB_CLIENT/MB_SERVER`, handles connection state, maps register image.	Keeps transport handling distinct from application logic.
`FB_InspectCoord`	Implements state machine, request issuance, acknowledgement handling, timeout logic, and result validation.	Encapsulates reusable coordination behavior.
`FB_WorkcellAdapter`	Connects machine-specific signals such as part-ready, cycle-complete, and reject handling to the generic coordinator.	Allows the same core integration block to be reused across stations.
`DB_Interface`	Stores the shared semantic fields exchanged over Modbus.	Makes the contract explicit and observable.
`DB_Diagnostics`	Stores counters, last fault, last request ID, and timing statistics if implemented.	Supports commissioning and maintenance.

This decomposition is consistent with Siemens and PLCopen guidance around reusable FB/DB structures [9, 12].

7.4 PLC coordination pseudocode

The core PLC-side logic can be expressed compactly in pseudocode. The actual IEC 61131-3 implementation may be written in SCL, FBD, or a combination, but the behavioral intent remains the same.

Algorithm 1 Proposed PLC coordination cycle

```
1: if state = IDLE and part_ready then
2:     req_id ← req_id + 1
3:     set req and publish current req_id
4:     start timeout timer
5:     state ← WAIT_ACK
6: end if
7: if state = WAIT_ACK and matching ack received then
8:     state ← WAIT_RESULT
9: else if state = WAIT_ACK and timeout or heartbeat loss then
10:    publish clear and reset
11:    record fault
12:    state ← RECOVERY
13: end if
14: if state = WAIT_RESULT and result_valid and result_id = req_id then
15:    accept result and release it to machine logic
16:    publish clear
17:    state ← RECOVERY
18: else if state = WAIT_RESULT and timeout or heartbeat loss then
19:    publish clear and reset
20:    record fault
21:    state ← RECOVERY
22: end if
23: if state = RECOVERY and interface synchronized then
24:    clear recovery flags
25:    state ← IDLE
26: end if
```

Two details deserve emphasis. First, result acceptance is gated by identifier equality, not by the mere presence of a result flag. Second, abnormal exits go through the same recovery structure instead of each fault type inventing its own ad hoc clean-up logic.

7.5 AI-side adapter behavior

The AI-side adapter has two operating modes in the case context: a simulation mode for controlled testing and a real mode intended to connect to the actual inspection software. The thesis treats this duality as a strength rather than a workaround. A

well-designed adapter should preserve the same external contract regardless of whether the upstream inspection engine is synthetic or real.

The essential AI-side behavior is:

1. poll or receive the current request-related registers;
2. if a new request identifier is observed, acknowledge it once and start processing;
3. when processing finishes, publish a valid result bound to that identifier;
4. if `clear` or `reset` is received, flush local state and return to idle;
5. toggle the heartbeat while alive.

Listing 7.1: Simplified AI-side adapter logic

```
loop:
    read Modbus register image

    if reset == 1:
        clear_local_state()
        publish_idle_state()
        continue

    if req == 1 and req_id not yet acknowledged:
        ack = 1
        ack_id = req_id
        busy = 1
        start inspection job for req_id

    if inspection job for req_id finished:
        result_valid = 1
        result_id = req_id
        result_code = PASS / FAIL / UNCERTAIN / ERROR
        busy = 0

    if clear == 1:
        clear_result_and_ack_fields()

    toggle heartbeat
end loop
```

The adapter is therefore kept intentionally simple. It should not infer workcell sequencing or try to correct PLC behavior automatically. Its role is to implement the interface contract correctly and consistently.

7.6 Baseline and proposed implementation contrast

The BASELINE design can be summarized as "request, wait, consume whatever result is visible, and drop the request on timeout." This is a reasonable first prototype because it minimizes implementation effort. It is also exactly the sort of design that appears attractive during early connectivity work.

The PROPOSED design adds just enough structure to make the interface industrially safe:

- request identifiers to correlate parts and results,
- acknowledgements to confirm request ownership,
- explicit result validity to separate partial from complete state,
- clear/reset transitions to flush interface state deterministically, and
- heartbeat supervision to distinguish silence from progress.

The implementation cost is modest because each of these additions is a small field or transition. The benefits become visible primarily under abnormal conditions, as shown later in the results chapter.

7.7 Instrumentation and logging

The evaluation required more than a qualitative impression that the line "seems to work". For every cycle, the software-in-the-loop environment logged the following timestamps or events:

- request issued,
- acknowledgment observed,
- result accepted,
- timeout triggered,

- restart or heartbeat-loss fault detected, and
- recovery completed.

From these events, end-to-end latency, success rates, timeout rates, unsafe outcome rates, and recovery behavior were computed. The same event model can be translated into PLC diagnostics and commissioning counters in a real deployment.

7.8 Implementation limits

Because the thesis centers on coordination and architecture, the PLC implementation is described at the level of reusable structure and behavioral logic rather than as a complete vendor project archive. The work remains faithful to Siemens block concepts and Modbus instruction patterns, but the main comparative evidence is generated in the software-in-the-loop environment. This is an acceptable methodological choice because the research problem concerns coordination semantics, not proprietary platform benchmarking.

Chapter 8

Experimental Setup and Evaluation Procedure

8.1 Evaluation objective

The evaluation was designed to answer a focused question: does the proposed semantic strengthening of the Modbus interface materially improve PLC–AI coordination under realistic abnormal conditions, and if so, at what cost in latency and complexity?

The objective is therefore comparative rather than absolute. The experiments are not intended to establish universal latency numbers for every Siemens installation. They are intended to expose behavioral differences between a simple shared-register baseline and an explicitly coordinated framework.

8.2 Test scenarios

Five single-station scenarios and one nominal scaling scenario were used, as summarized in Table 8.1.

Table 8.1: Experimental scenarios used in the evaluation.

Scenario	Injected condition	Purpose
Nominal	Regular AI processing delay with no intentional faults.	Establish baseline latency and jitter under ordinary operation.
Delayed AI	A subset of cycles receives significantly longer processing delay.	Test timeout sensitivity and resilience to AI timing variability.
Communication fault	Random poll drops are injected.	Evaluate robustness to intermittent transport disturbance.
AI restart	The AI-side process may restart during active work.	Test liveness supervision and restart-safe recovery.
Stale result	Some results are intentionally delayed beyond the timeout window.	Expose stale-result acceptance risk.
Scaling	Multiple virtual stations share one polling client under nominal processing.	Evaluate the effect of serialized polling on latency and completion rates.

Each single-station scenario was executed for 600 cycles per architecture. The scaling scenario used 400 cycles for each station count (1, 2, 4, and 8 stations).

8.3 Injected delay and fault model

The nominal scenario uses a moderately distributed AI processing delay. The delayed AI scenario increases both the average delay and the long-tail probability. The communication fault scenario injects random poll drops. The AI restart scenario permits the inspection process to restart while a request is active. The stale result scenario intentionally generates a subset of inspection completions beyond the timeout threshold.

This fault model was designed to mirror the conceptual problems identified in the project material and literature review rather than to emulate one specific production line. Its purpose is to expose the differences between architectural choices.

8.4 Outcome categories

Every cycle is classified into one of the following outcomes:

- **Success**: a correct result for the current request is accepted;
- **Timeout**: the PLC reaches its timeout limit without a valid accepted result;
- **Stale acceptance**: the PLC accepts a result that belongs to an earlier request;
- **Restart fault**: the PLC explicitly detects a heartbeat-loss or restart-related condition.

The stale-acceptance category is the most important unsafe outcome because it represents silent semantic failure.

8.5 Analytical recovery bound

In addition to measured cycle statistics, the thesis reports a configuration-derived recovery bound. This bound is not a measured physical constant. It is a conservative analytical value derived from the chosen timeout, poll visibility, and recovery settings in the model. Its purpose is to illustrate that a framework with explicit liveness and reset semantics can be configured for tighter recovery than one that relies on implicit cleanup.

8.6 Scaling procedure

For the scaling scenario, the proposed architecture was evaluated while increasing the number of virtual stations sharing one polling client. This experiment intentionally isolates one design question: how far can a serialized polling loop be pushed before latency and successful completion degrade under unchanged timeout settings?

This is not the only possible scaling architecture. A production deployment may use multiple client instances, staggered polling groups, or station-local buffering. The experiment therefore evaluates a specific but common scaling pattern rather than claiming an upper bound for all implementations.

8.7 Analysis procedure

The analysis proceeded in four steps.

1. Aggregate cycle-level data into scenario summaries.
2. Compare baseline and proposed variants on success, timeout, unsafe outcomes, and latency distribution.
3. Interpret the results in relation to the research questions and the literature-derived requirements.
4. Extract architectural implications, especially where correctness and latency trade off against one another.

Chapter 9

Results

9.1 Overview

This chapter reports the quantitative outcomes from the nominal, delayed AI, communication fault, stale result, AI restart, and scaling experiments. The single-station scenarios used 600 cycles per architecture, and the scaling scenario used 400 cycles for each station count, as described in chapter 8.

9.2 Nominal performance

Under nominal conditions, both architectures completed all 600 cycles successfully. The p95 successful-cycle latency was 180 ms for both designs. Mean successful-cycle latency was 143.0 ms for the baseline and 143.3 ms for the proposed framework. The jitter metric was 20.6 ms for the baseline and 22.2 ms for the proposed framework.

Figure 9.1 shows the latency distributions for successful cycles in the nominal and delayed-AI scenarios. The median line in the boxplot corresponds to the p50 latency reference, while p95 remains the main tabulated response-time statistic for the single-station scenarios.

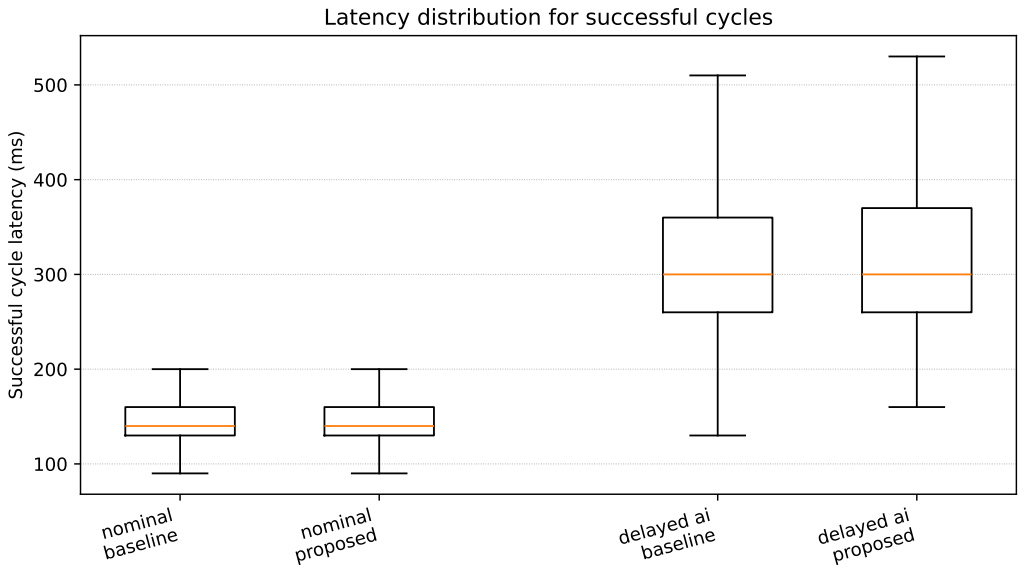


Figure 9.1: Latency distribution for successful cycles in the nominal and delayed-AI scenarios.

9.3 Delayed-AI scenario

In the delayed-AI scenario, the baseline completed 87.7% of cycles successfully, timed out on 6.5%, and accepted stale results in 5.8% of cycles. The proposed design completed 94.7% successfully, timed out on 5.3%, and had 0% stale-result acceptance.

The successful-cycle p95 latency was 550 ms for the baseline and 560 ms for the proposed framework. The jitter metric was 104.1 ms for the baseline and 108.5 ms for the proposed framework.

9.4 Communication-fault scenario

In the communication-fault scenario, intermittent poll losses were injected. The baseline completed 100.0% of cycles successfully, whereas the proposed design completed 98.3% successfully and classified 1.7% of cycles as timeouts. Neither design produced stale acceptance in this scenario. The p95 successful-cycle latency was 220 ms for both designs, with jitter values of 28.0 ms for the baseline and 29.4 ms for the proposed framework.

9.5 Stale-result scenario

In the stale-result scenario, some inspection results were intentionally delayed beyond the timeout window so that they remained visible after the PLC had already advanced to a new cycle.

Under these conditions, the baseline completed 71.3% of cycles successfully, timed out on 14.5%, and accepted stale results in 14.2% of cycles. The proposed framework completed 84.3% successfully, timed out on 15.7%, and had 0% stale acceptance. The p95 successful-cycle latency was 200 ms for the baseline and 210 ms for the proposed framework.

9.6 AI-restart scenario

The AI-restart scenario represents loss of liveness during active processing. The baseline completed 38.0% of cycles successfully, timed out on 39.2%, and accepted stale results in 22.8% of cycles. The proposed framework completed 38.7% successfully, reduced timeouts to 21.2%, had 0% stale acceptance, and classified 40.2% of cycles as restart faults.

9.7 Selected scenario metrics

Table 9.1 lists the selected metrics for the nominal, delayed-AI, communication-fault, and stale-result scenarios.

Table 9.1: Selected metrics for nominal, delayed, communication-fault, and stale-result scenarios. Latency values refer to successful cycles only.

Scenario	Arch.	Success (%)	Timeout (%)	Unsafe (%)	p95 latency (ms)	Jitter (ms)
Nominal	Baseline	100.0	0.0	0.0	180	20.6
Nominal	Proposed	100.0	0.0	0.0	180	22.2
Delayed AI	Baseline	87.7	6.5	5.8	550	104.1
Delayed AI	Proposed	94.7	5.3	0.0	560	108.5
Comm. fault	Baseline	100.0	0.0	0.0	220	28.0
Comm. fault	Proposed	98.3	1.7	0.0	220	29.4
Stale result	Baseline	71.3	14.5	14.2	200	25.6
Stale result	Proposed	84.3	15.7	0.0	210	26.5

9.8 Restart detection and bounded recovery

Figure 9.2 reports the configuration-derived recovery bound used in the evaluation model.

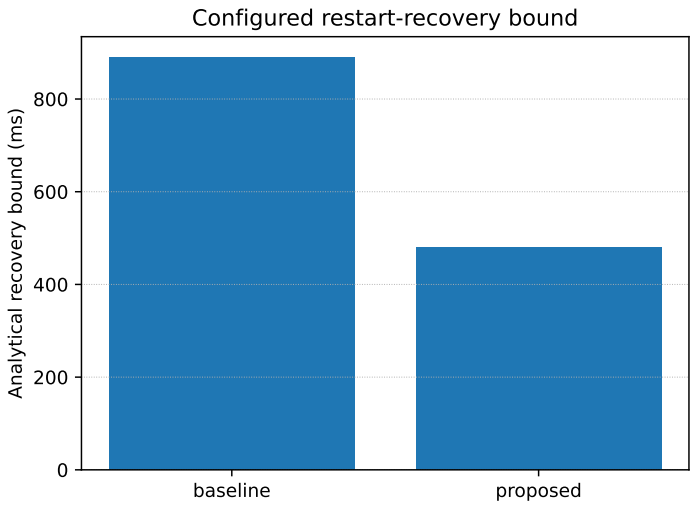


Figure 9.2: Configuration-derived recovery bound in the evaluation model.

Table 9.2 lists the outcome breakdown for the AI-restart scenario.

Table 9.2: Outcome breakdown for the AI-restart scenario.

Scenario	Arch.	Success (%)	Timeout (%)	Unsafe (%)	Restart fault (%)	p95 latency (ms)
AI restart	Baseline	38.0	39.2	22.8	0.0	560
AI restart	Proposed	38.7	21.2	0.0	40.2	240

9.9 Scaling results

The nominal scaling experiment evaluated the proposed architecture with 1, 2, 4, and 8 virtual stations sharing one polling client. The p95 successful-cycle latency was 180 ms, 200 ms, 220 ms, and 290 ms as station count increased. Successful completion was 100.0% for one and two stations, 50.0% at four stations, and 25.5% at eight stations when timeout settings were left unchanged. Figures 9.3 and 9.4 plot the latency and success-rate trends, and Table 9.3 lists the corresponding values, including p99 as a tail-latency check for the serialized polling experiment.

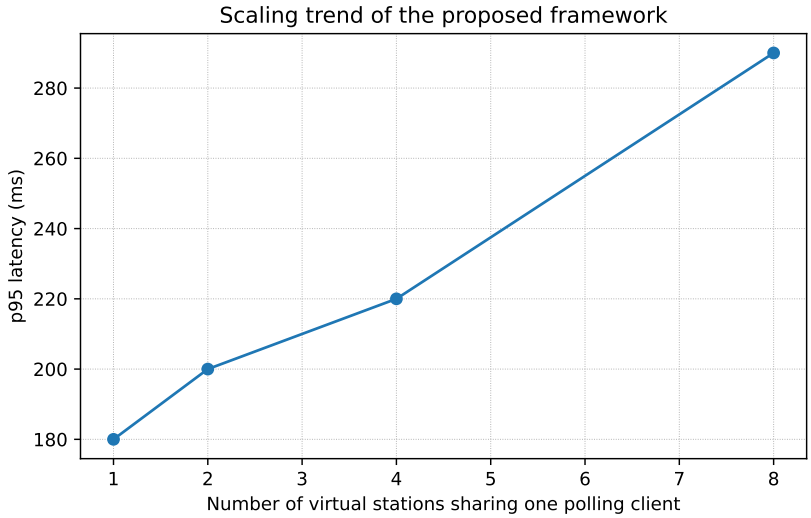


Figure 9.3: Successful-cycle p95 latency as the number of stations sharing one polling client increases.

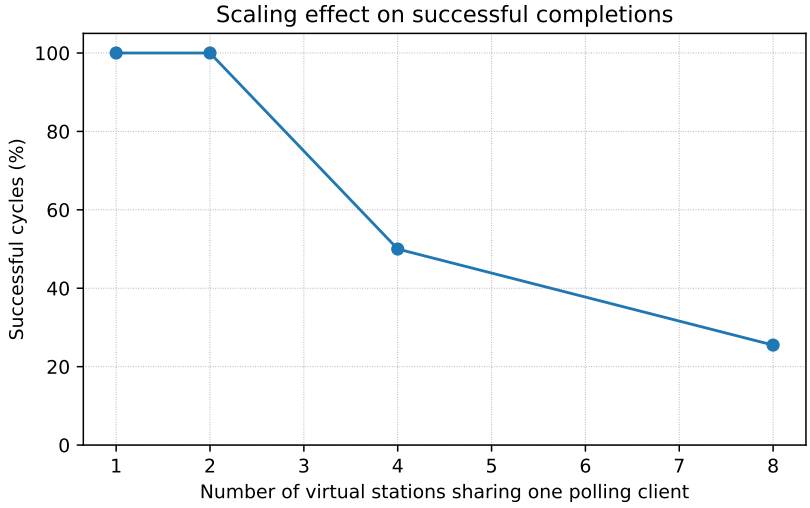


Figure 9.4: Successful cycle completion as the number of stations sharing one polling client increases.

Table 9.3: Scaling results for the proposed architecture under nominal processing delay.

Stations	Success (%)	Timeout (%)	p95 latency (ms)	p99 latency (ms)
1	100.0	0.0	180	190
2	100.0	0.0	200	210
4	50.0	50.0	220	280
8	25.5	74.5	290	300

Chapter 10

Discussion

10.1 Interpreting the main result

The central finding of the thesis is not that a richer handshake makes Modbus faster. It is that a modest increase in semantic structure makes Modbus-based PLC coordination markedly safer and more reusable for AI-enabled inspection. The nominal result supports this interpretation: both architectures reached a p95 successful-cycle latency of 180 ms, so the proposed semantic fields did not create a meaningful nominal latency penalty in the model.

Table 9.1 should therefore not be read primarily as a latency comparison. The strongest contrast is the unsafe-outcome column. In the delayed-AI case, the proposed design accepted a small p95 latency increase from 550 ms to 560 ms while reducing stale-result acceptance from 5.8% to 0%. This is a coordination trade-off rather than a speed optimization.

This matters because early integration work is often judged by nominal demonstrations. A small set of shared registers can appear sufficient when the demo succeeds. What such demonstrations typically hide, however, are stale-result risk, restart ambiguity, and the absence of clear ownership over a request. The experiments show that these weaknesses become visible precisely in the delayed, timed-out, and restarted conditions that make real deployments difficult.

10.2 What the thesis says about PLCs as coordination layers

The answer to **RQ1** is therefore not simply yes or no. A Siemens S7-1200 PLC can be a reliable coordination and communication layer for AI-based inspection, but this reliability does not come from the PLC by itself. It depends on how the PLC is used for coordination. If the PLC only transfers bits between systems, deterministic execution cannot protect the workcell from semantic errors. However, if the PLC controls the request lifecycle, timeout policy, and rules for accepting results, it can become an effective integration boundary.

This interpretation aligns with the broader PLC software-structuring literature [10,13]. The PLC is valuable not only because it executes deterministically, but because it is the natural place to encode sequence authority in a maintainable form.

10.3 What the thesis says about Modbus TCP

The answer to **RQ2** is that Modbus TCP is adequate and sensible for the studied integration path, especially because it already fits the Siemens/*NorrSpect* baseline and remains straightforward to deploy. The results support the view from Tapia et al. that Modbus can be competitive on latency while remaining relatively simple from a software perspective [16]. At the same time, the thesis makes clear that Modbus does not solve the semantic problem by itself.

This is the key design lesson about protocol choice in the present case. The protocol is not inadequate because it lacks a rich object model; it becomes inadequate only when the integration team assumes that register transport is equivalent to coordination. For a limited cell-level interface with a modest payload size, Modbus can still work well. However, the main engineering responsibility is moved to the application contract.

10.4 Why request identifiers matter so much

The experiments show that request identifiers are the most important improvement in the framework. They prevented stale results from being accepted after timeouts and delayed responses. Without request identifiers, the PLC must decide whether a result is correct mainly from timing and flag states. This is not reliable enough, because these signals alone can be misleading.

The stale-result scenario is the clearest example. A timeout is visible and recoverable, while stale acceptance can remain silent. By binding every accepted result to a request identifier, the proposed design removed the tested stale-acceptance failure mode.

In practical industrial terms, the request identifier plays the same role as a part ticket or transaction reference. It is a small mechanism with a disproportionately large safety payoff. The lesson is not limited to Modbus. Any PLC–AI integration that lacks explicit request/result correlation is exposed to the same class of semantic error.

10.5 Timeouts, faults, and engineering honesty

The communication-fault scenario needs careful interpretation. Viewed only by success percentage, the baseline appears better because it completed all cycles while the proposed design classified 1.7% of cycles as timeouts. That comparison is incomplete because the baseline does not require the same explicit acknowledgement and correlation before accepting a cycle as successful. The proposed design is more conservative when coordination guarantees cannot be maintained.

A useful insight from the AI-restart scenario is that success rate is not always the most informative headline metric. In that scenario, the proposed design did not rescue most interrupted workpieces. What it did instead was more valuable: it transformed ambiguous behavior into explicit fault classification. The reduction of p95 successful latency from 560 ms to 240 ms in the restart scenario should not be read as faster inference; it mainly reflects earlier fault classification instead of extended waiting for a response that may never arrive.

Put differently, the proposed architecture is willing to fail fast and visibly when trust in the result is lost. For industrial automation, that is often preferable to optimistic waiting or silent misclassification. The framework therefore embodies an engineering preference for *safe explicit failure* over *implicit apparent continuity*.

10.6 Where PLC-based coordination is sufficient

The thesis suggests that PLC-based coordination is sufficient when the integration problem has the following characteristics:

- the PLC remains the owner of machine sequence and interlocks;
- the inspection payload can be reduced to a bounded set of values such as result codes, identifiers, and a few metadata fields;

- the station count is moderate or the communication architecture is partitioned appropriately; and
- timeouts and restart conditions can be handled through bounded recovery logic rather than human improvisation.

This describes many cell-level industrial inspection cases well. In such settings, the PLC is a good coordination anchor because it already sits at the interface between machine logic and external services.

10.7 Where the limits begin

The scaling experiment and the wider literature suggest where limits appear.

First, one polling client that handles requests one after another does not scale well when several stations must share the same timeout limit. The scaling results do not invalidate the framework; they identify a clear architectural boundary. The main issue is not the handshake itself, but the communication topology that was chosen.

Second, larger inspection payloads, such as large trace records, image references, or detailed semantic metadata, are less suitable for a compact register-based protocol.

Third, cybersecurity becomes more important when the interface is used outside trusted lab or cell networks. In these situations, plain Modbus TCP is not sufficient, even if the coordination logic is otherwise correct [18].

Fourth, safety-certified behavior remains outside the scope of the framework. The thesis improves functional coordination, but it does not make inspection-based decisions safety-rated.

10.8 Comparison with prior literature

The discussion above helps explain how this thesis complements earlier work.

Compared with the Industry 4.0 and CPS literature, the thesis moves down from architectural aspiration to coordination mechanics [1–3].

Compared with protocol comparison studies, this thesis focuses less on classifying protocols and more on the semantic layer needed for PLC–AI interaction [15, 16].

Compared with AI inspection research, it shifts emphasis from model performance to the automation boundary that determines whether the model can be used safely in a

line [4, 5].

Compared with recent PLC–vision case studies, it contributes a more explicit discussion of stale-result prevention, restart handling, and reusable interface semantics [6, 7].

10.9 Practical implications for NorrSpect-style deployments

For *NorrSpect* and similar industrial software suppliers, the practical message is direct. The term *NorrSpect*-style is used here to describe an external AI inspection service integrated with PLC-controlled equipment, not to claim that *NorrSpect* is either uniquely difficult or fully representative of every vendor. The most reusable asset in this type of integration is not a line-specific register map, but a versioned interface contract paired with a PLC-side coordination pattern.

That implies at least five concrete practices:

1. standardize the request/result contract across customer projects;
2. include request identifiers by default, not only after problems appear;
3. expose heartbeat and diagnostic counters from the first prototype;
4. keep machine sequencing in the PLC and inspection semantics in the AI system; and
5. scale by instantiating modular coordination blocks rather than duplicating custom logic.

These practices are not difficult to adopt, but they make a meaningful difference to maintainability and rollout speed.

10.10 Limitations

The thesis has five main limitations.

First, the core evaluation is software-in-the-loop rather than a large physical hardware campaign. This strengthens repeatability and fault injection, but weakens claims about exact device-level timing.

Second, the simulated AI block controlled delay, restart, and stale-result behavior, but it did not model unconsidered software failures, wrong classifications, camera or image-acquisition faults, or plant-level disturbances. The reliability of a deployed *NorrSpect* integration therefore still needs hardware-in-the-loop and production-oriented validation.

Third, the work uses a bench-informed integration model rather than a full production deployment with real customer data.

Fourth, the scope deliberately excludes safety certification and detailed cybersecurity engineering.

Fifth, only one primary protocol path—Modbus TCP—was developed in depth, following the approved project focus. This is appropriate for the case, but it means the thesis is strongest as a Modbus-centered study rather than a universal protocol-comparison thesis.

Chapter 11

Conclusion and Future Work

11.1 Conclusion

This thesis set out to design, implement, and evaluate a modular PLC integration framework for AI-based industrial inspection software, centered on a Siemens S7-1200 and Modbus TCP integration path relevant to *NorrSpect*. The work was motivated by a clear industrial problem: although AI-based inspection is increasingly valuable in production systems, its integration with PLC-controlled automation remains too often ad hoc, fragile, and difficult to reuse.

The thesis showed that the key challenge is not protocol connectivity alone, but coordination semantics. A deterministic PLC can absorb non-deterministic AI behavior only when the integration contract makes ownership, timing, and recovery explicit.

The resulting artifact strengthened the baseline Modbus approach with request identifiers, acknowledgements, explicit result validity, clear/reset handling, heartbeat monitoring, and a compact PLC-side state machine. Comparative evaluation showed that this design preserved nominal latency while materially improving behavior under delayed-AI, stale-result, and restart conditions. The most important improvement was the elimination of unsafe stale-result acceptance in the abnormal scenarios that were tested.

11.2 Answers to the research questions

RQ1: To what extent can a Siemens S7-1200 PLC act as a reliable coordination and communication layer between *NorrSpect* AI-based inspection software and industrial automation equipment?

Within the integration path studied here for *NorrSpect*, Siemens S7-1200, and Modbus TCP, a Siemens S7-1200 PLC can serve as a reliable coordination layer for AI-based inspection, provided that the PLC owns request lifecycle, timeout enforcement, and result acceptance semantics.

RQ2: How does Modbus TCP perform in the implemented S7-1200 integration path with respect to latency, jitter, robustness, recovery behavior, and scalability?

Modbus TCP worked well enough for the intended cell-level integration task. It provided low latency during normal operation and was simple to implement. However, because the protocol is simple, correctness depends mainly on the application-level semantics rather than on advanced protocol features.

RQ3: How effectively can standardized handshakes and state-machine-based coordination manage AI-induced timing variation, communication delays, and fault scenarios over Modbus TCP?

Standardized handshakes and state-machine-based coordination proved effective. They reduced unsafe outcomes, improved diagnosability, and turned restart-related ambiguity into explicit faults.

RQ4: What architectural design principles can be generalized from the Siemens/Modbus testbed into a reusable PLC-side integration framework?

The most important design principles that can be reused are to separate transport from semantics, connect every result to its request, make recovery behavior explicit, provide observability for faults and states, and treat scaling as a communication-architecture issue rather than a mere duplication exercise.

11.3 Future work

Several extensions would strengthen the work further.

A first priority is hardware-in-the-loop validation on the actual Siemens bench with the finalized PLC blocks and the real *NorrSpect* adapter. This would allow direct comparison between the software-in-the-loop findings and measured PLC/communication behavior. That validation should include unplanned AI-side and adapter faults, not only the scripted delays and restarts used in the simulator.

A second priority is multi-station architecture improvement. The scaling results suggest testing multiple Modbus clients, partitioned polling groups, or buffered station adapters.

A third priority is adding deployment-grade security, whether through segmented networks, secure tunnels, or the Modbus security profile where appropriate.

A fourth priority is integrating formal or semi-formal verification of the coordinator state logic, building on the wider literature around PLC software quality and verification.

11.4 Final contribution statement

This thesis gives a practical and technically supported answer to a specific but important problem in modern automation: how a PLC can coordinate with external AI inspection software through a simple industrial protocol without reducing trustworthiness. The framework is not a complete industrial product, but it offers a reusable and academically justified integration pattern. This pattern can support future *NorrSpect* deployments and other PLC–AI integration work.

Appendix A

Current and Proposed Register Maps

This appendix restates the baseline and proposed register maps as a compact lookup reference. The proposed map intentionally mirrors the semantic contract in Table 6.2, but uses shorter deployment-oriented wording.

A.1 Current baseline map

Table A.1 restates the documented baseline interface in compact form.

Table A.1: Baseline register map (documented starting point).

Registers	Meaning
0–9	Status, pieces inspected, batch identifier, serial number, defect rate, reserved values.
10–19	Machine status bits and reserved values. Register 10 includes Stop, Emergency Stop, and Ready bits.

A.2 Proposed semantic map

Table A.2 restates the proposed semantic map for quick reference.

Table A.2: Proposed semantic register map.

Register	Symbol	Meaning
0	`req`	New request present.
1	`req_id`	Current request identifier.
2	`clear`	Clear current result and acknowledgment state.
3	`reset`	Force re-synchronization after abnormal behavior.
4	`hb_plc`	PLC heartbeat.
10	`ack`	Request acknowledged by AI side.
11	`ack_id`	Identifier of acknowledged request.
12	`busy`	AI side is processing the current request.
13	`result_valid`	Result fields contain a valid outcome.
14	`result_id`	Identifier of the request to which the result belongs.
15	`result_code`	PASS/FAIL/UNCERTAIN/ERROR.
16	`confidence`	Optional confidence score.
17	`hb_ai`	AI heartbeat.
18	`fault_code`	Explicit abnormal condition code.
19	Reserved	Reserved for deployment-specific extensions.

Appendix B

PLC State Transition Summary

Table B.1 summarizes the main transitions used by the proposed PLC coordinator.

Table B.1: State transition summary for the proposed PLC coordinator.

Current state	Condition	Next state	Main action
IDLE	Part ready	WAIT_ACK	Publish `req` and new `req_id`.
WAIT_ACK	Matching acknowledgment	WAIT_RESULT	Keep request active and wait for result.
WAIT_ACK	Timeout or heartbeat loss	RECOVERY	Publish `clear` and `reset`; record fault.
WAIT_RESULT	Matching valid result	RECOVERY	Accept result and publish `clear`.
WAIT_RESULT	Timeout or heartbeat loss	RECOVERY	Publish `clear` and `reset`; record fault.
RECOVERY	Interface synchronized	IDLE	Clear transient flags and arm next cycle.

Appendix C

Experiment Matrix

Table C.1 lists the experimental matrix used for the comparative evaluation.

Table C.1: Experiment matrix used in the thesis.

Scenario	Architecture	Fault/variation pattern	Repetitions
Nominal	Baseline and Proposed	Standard processing-delay distribution	600 each
Delayed AI	Baseline and Proposed	Long-tail inference delay injected on subset of cycles	600 each
Communication fault	Baseline and Proposed	Random poll-drop behavior	600 each
AI restart	Baseline and Proposed	Mid-process restart events and heartbeat loss	600 each
Stale result	Baseline and Proposed	Delayed result beyond timeout window	600 each
Scaling	Proposed only	1, 2, 4, and 8 stations share one polling client	400 per station count

Bibliography

[1] L. Monostori, "Cyber-physical production systems: Roots, expectations and R&D challenges," *Procedia CIRP*, vol. 17, pp. 9–13, 2014.

[2] J. Lee, B. Bagheri, and H.-A. Kao, "A cyber-physical systems architecture for Industry 4.0-based manufacturing systems," *Manufacturing Letters*, vol. 3, pp. 18–23, 2015.

[3] A. W. Colombo, S. Karnouskos, O. Kaynak, Y. Shi, and S. Yin, "Industrial cyberphysical systems: A backbone of the fourth industrial revolution," *IEEE Industrial Electronics Magazine*, vol. 11, no. 1, pp. 6–16, 2017.

[4] S. Sundaram and A. Zeid, "Artificial intelligence-based smart quality inspection for manufacturing," *Micromachines*, vol. 14, no. 3, p. 570, 2023.

[5] D. Y. Patrashko and V. Gurau, "Machine learning-powered vision for robotic inspection in manufacturing: A review," *Sensors*, vol. 26, no. 3, p. 788, 2026.

[6] M. Maślanka, D. Jancarczyk, and J. Ryśiński, "Integration of machine vision and PLC-based control for scalable quality inspection in Industry 4.0," *Sensors*, vol. 25, no. 20, p. 6383, 2025.

[7] N. Almtireen, V. Reddy, M. Sutton, A. Nedvidek, C. Karn, M. Ryalat, H. Elmoaqet, and N. Rawashdeh, "PLC-controlled intelligent conveyor system with AI-enhanced vision for efficient waste sorting," *Applied Sciences*, vol. 15, no. 3, p. 1550, 2025.

[8] International Electrotechnical Commission, *Programmable Controllers – Part 3: Programming Languages*, 3rd ed., International Electrotechnical Commission, Geneva, Switzerland, 2013, iEC 61131-3.

[9] Siemens AG, "How do you program the PLC with STEP 7 (TIA Portal) in compliance with the IEC 61131-3 standard?" Siemens Industry Online Support FAQ ID 50204938, 2013, accessed via Siemens Industry Online Support.

[10] PLCopen, “Structuring program development with IEC 61131-3,” Technical guideline, 2018, PLCopen.

[11] ——, “Structuring with SFC: Do’s and don’ts,” Technical guideline, 2018, PLCopen.

[12] ——, “Software construction guidelines,” Technical guideline, 2018, PLCopen.

[13] B. Vogel-Heuser, J. Fischer, S. Feldmann, S. Ulewicz, and S. Rösch, “Modularity and architecture of PLC-based software for automated production systems: An analysis in industrial companies,” *Journal of Systems and Software*, vol. 131, pp. 35–62, 2017.

[14] Modbus Organization, *MODBUS Application Protocol Specification V1.1b3*, Modbus Organization, Apr. 2012, April 26, 2012.

[15] S. Jaloudi, “Communication protocols of an industrial internet of things environment: A comparative study,” *Future Internet*, vol. 11, no. 3, p. 66, 2019.

[16] E. Tapia, L. Sastoque-Pinilla, U. Lopez-Novoa, I. nigo Bediaga, and N. L. de Lacalle, “Assessing industrial communication protocols to bridge the gap between machine tools and software monitoring,” *Sensors*, vol. 23, no. 12, p. 5694, 2023.

[17] Siemens AG, “MODBUS-TCP using the instructions MB_CLIENT and MB_SERVER for SIMATIC S7-1200/S7-1500,” Application example, document ID 83130159, 2016, Siemens Industry Online Support.

[18] Modbus Organization, *MODBUS/TCP Security Protocol Specification*, Modbus Organization, 2021, Version 3.6, July 30, 2021.

[19] M. Wollschlaeger, T. Sauter, and J. Jasperneite, “The future of industrial communication: Automation networks in the era of the internet of things and Industry 4.0,” *IEEE Industrial Electronics Magazine*, vol. 11, no. 1, pp. 17–27, 2017.

[20] T. Reichenstein, T. Raffin, C. Sand, and J. Franke, “Implementation of machine vision based quality inspection in production: An approach for the accelerated execution of case studies,” *Procedia CIRP*, vol. 112, pp. 596–601, 2022.

[21] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[22] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[23] B. Flyvbjerg, “Five misunderstandings about case-study research,” *Qualitative Inquiry*, vol. 12, no. 2, pp. 219–245, 2006.